CracksLatinoS! 2006

-* UnPackMe_UnOpix 1.10*-

Programa	UnPackMe_UnOpix 1.10
Download	FTP de Ricardo
Descripción	Packer
Herramientas	OllyDbg Parcheado4 + PETools RC7 + ImpREC 1.6
Dificultad	A ver.. es fácil si das con la tecla, jeje..
Compresor/Compilador	UnOpix 1.10/FASM
Protección	Packer
Objetivos	Desempacar manualmente
Cracker	SyXe'05 **Fecha:** 11/11/06
Tutorial n°	5° concurso.

-** Introduccion **-

Buenas a todos los CracksLatinos, nos volvemos a ver las caras aunque sea a través de un texto, jeje.. pero eso está bien, buena señal ☺

En esta ocasión nos vamos a centrar en los concursos que venimos haciendo, justamente en el concurso 109 en su tercer nivel (nivel3) que es el que me ha tocado hacer el tute.

Por todo ello.. aquí estamos, je

-** Explorando el objetivo **-

Sabiendo esto sólo nos queda echarle un vistazo a la víctima con PEiD 0.94 a ver si nos reconoce lo que tiene:

Nothing found *

Nada no lo reconoce, miremos con RDG Packer 0.6.5 Beta a ver si lo reconoce, en Modo M-A nos dice absolutamente NADA, o sea se queda la pantallita en blanco.

Pues vale, vamos a cambiarle el RadioButton al Modo M-B y después pulsamos en DETECTAR a ver si tenemos más suerte:

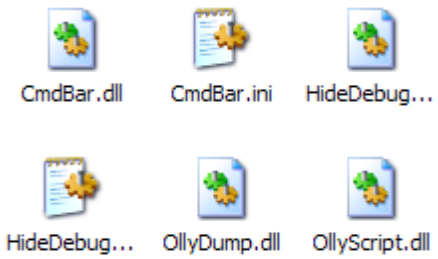
Pues tampoco, nos dice el lenguaje en que fue programado el crackme (UnPackMe), pero bueno en cualquier caso nos fiaremos de Ricardo y si allí dice que es un UnOpix pues será UnOpix!

-** Desempacando **-

Pues bien, llegó el momento más esperado del tute "el desempakado" en este momento ya tengo el dumpeado, pero voy a hacer como Ricardo en la parte 56, es decir, iré tratando de contar todo lo que he ido haciendo hasta llegar a la solución final, mostrando los hostiazos que me he llevao y to eso, je..

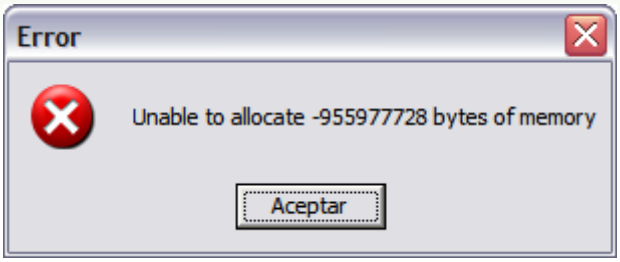
Los que han leído tutes anteriores saben que personalmente me encanta escribir a la par que realizo el trabajo, pero ¿porqué? Pues porque así se muestra todo el camino recorrido desde el comienzo hasta el final, quedando un tute de puta madre y que cualquiera puede seguir sin dificultad, además de mostrar métodos e ideas que se le van ocurriendo a uno sobre la marcha. En este caso haremos eso mostrar todo o gran parte del camino recorrido hasta llegar a la solución FINAL sip? Pues ale palante :D

Cerramos ambos analizadores (PEiD y RDG) y abrimos un Olly, de momento con muy pocos plugins:

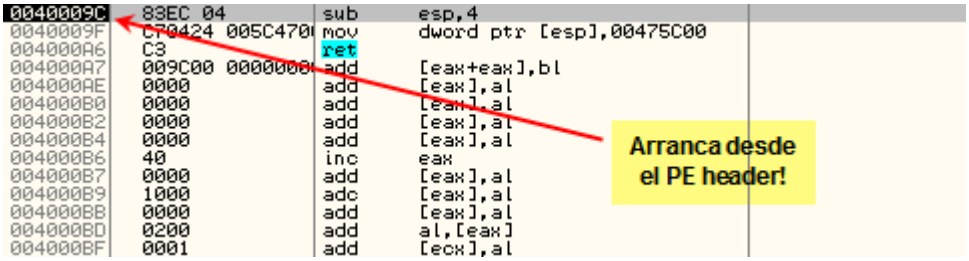


De momento iremos con estos, si luego necesitaramos alguno más pues se lo ponemos y seguiríamos. Recordad que estamos trabajando con un Olly modificado por el maestro Ricardo llamado Parcheado4 que evita el bug de "Illegal Instruction" no sé si será necesario para este packer, pero por si acaso nos curamos en salud, y digo esto porque si alguien intenta seguir el tute que tenga en cuenta con qué Olly estoy trabajando aquí sip?

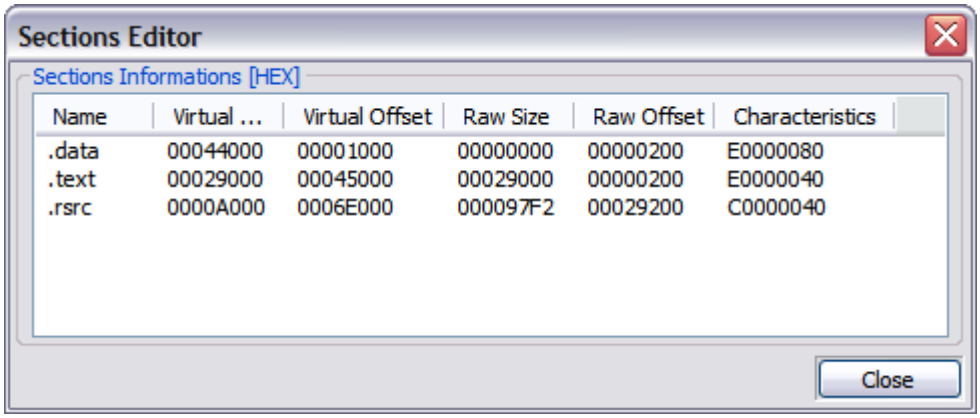
Bueno he de decir que en mi máquina antes de llegar al EP sale esto:



Joer! No puede ubicar en memoria un número negativo?? de bytes, jejejeje.. lo nunca visto pero bueno, lo aceptamos y para en el EP:



Vale, más cosas raras, de entrada el EntryPoint no está en la sección del packer como sería lo normal sino en el PE header (cabecera) del archivo ejecutable, con PE Tools desde TOOLS-PE EDITOR, seleccionamos el UnPackMe y en la siguiente pantallita apretamos el botón SECTIONS y ya podemos ver sus secciones:

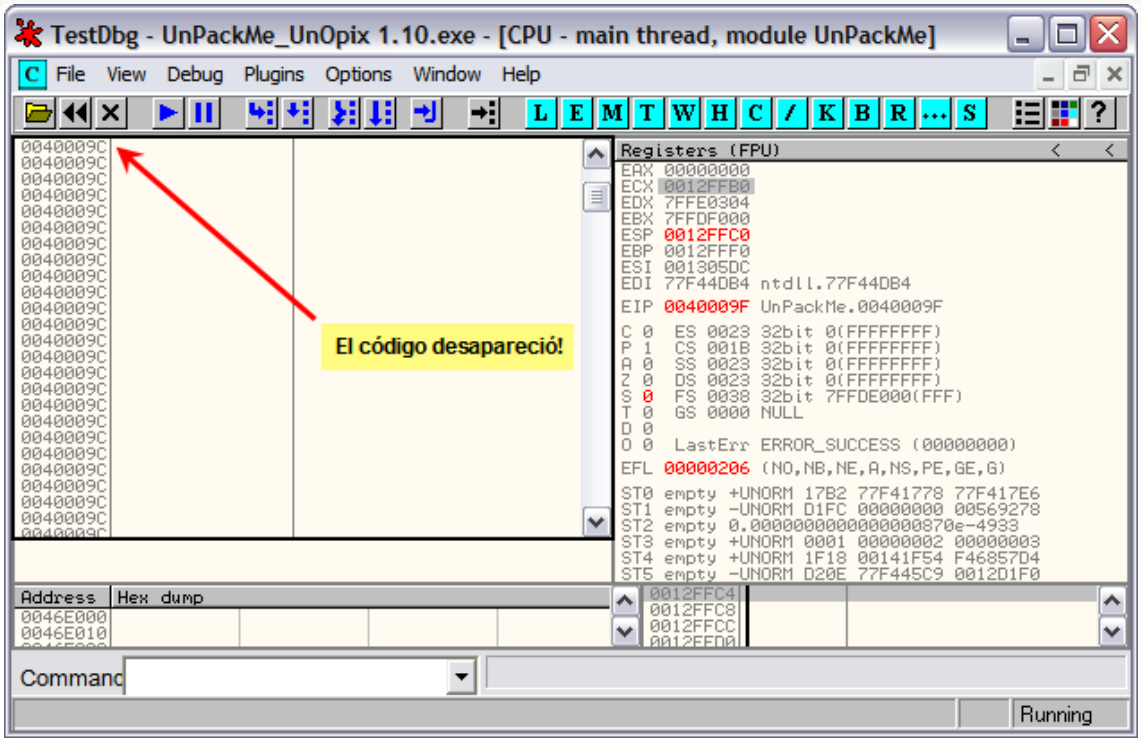


Name	Virtual ...	Virtual Offset	Raw Size	Raw Offset	Characteristics
.data	00044000	00001000	00000000	00000200	E0000080
.text	00029000	00045000	00029000	00000200	E0000040
.rsrc	0000A000	0006E000	000097F2	00029200	C0000040

Sabemos que los editores no suelen mostrar la sección del header (cabecera) que ocupa los primeros 0x1000h bytes del archivo, y es allí desde donde arranca el crackme empakado, otra curiosidad es que vemos que está antes la sección .data que la .text, normalmente esta última es la que alberga el código enecutable del archivo, peeeeeeero como todos sabemos se le puede dar el nombre que queramos a las secciones, por lo tanto no le damos valor a este detalle, solo es una curiosidad.

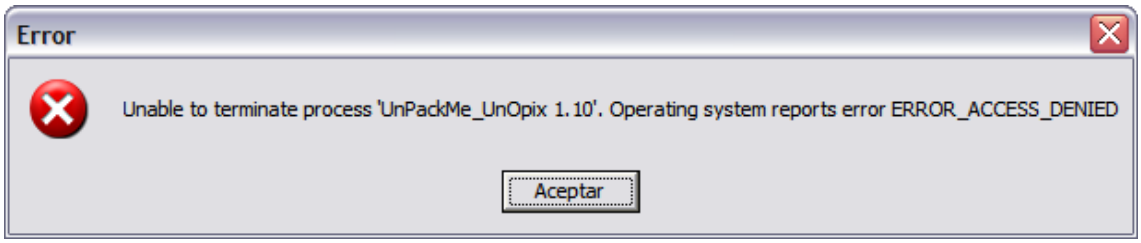
Bien sigamos, si damos RUN en Olly (F9) vemos que no corre

Quedó RUNNING, hasta aquí todo correcto pero si esperamos unos segundos en lugar de aparecer el UnPackMe (crackme de aquí en adelante) vemos que quedó como congelado, si esperamos unos segundos se nos queda la cara de bobo al ver que no hay código¡!!!!!!!!!



Pues vaya que bien! Volveremos a probar, pero esta vez mientras queda RUNNING pausamos Olly con F12 y... ocurre lo mismo! joer pues si que va a dar guerra esto, grrrrrrr.. tampoco hay datos en el Dump ni en el snack (pila) ??

Si probamos a recargar con CTRL+F2 nos tira otro error diciendo algo así como "Unable to terminate process "UnPackMe_UnOpix 1.10. Operating system reports error ERROR_ACCESS_DENIED". O sea, que además de no poder correrlo no puede ni cerrarlo, je.. y en el cartel nos dice eso mismo que no lo puede cerrar y que el propio Sistema Operativo (S.O) se encargará de generar un informe del mismo.



La forma de cerrar Olly será a la Brava, picando en el Aspa de la X o bien con ALT+X y aun así nos tira el mismo error pero se termina cerrando el crackme y Olly.

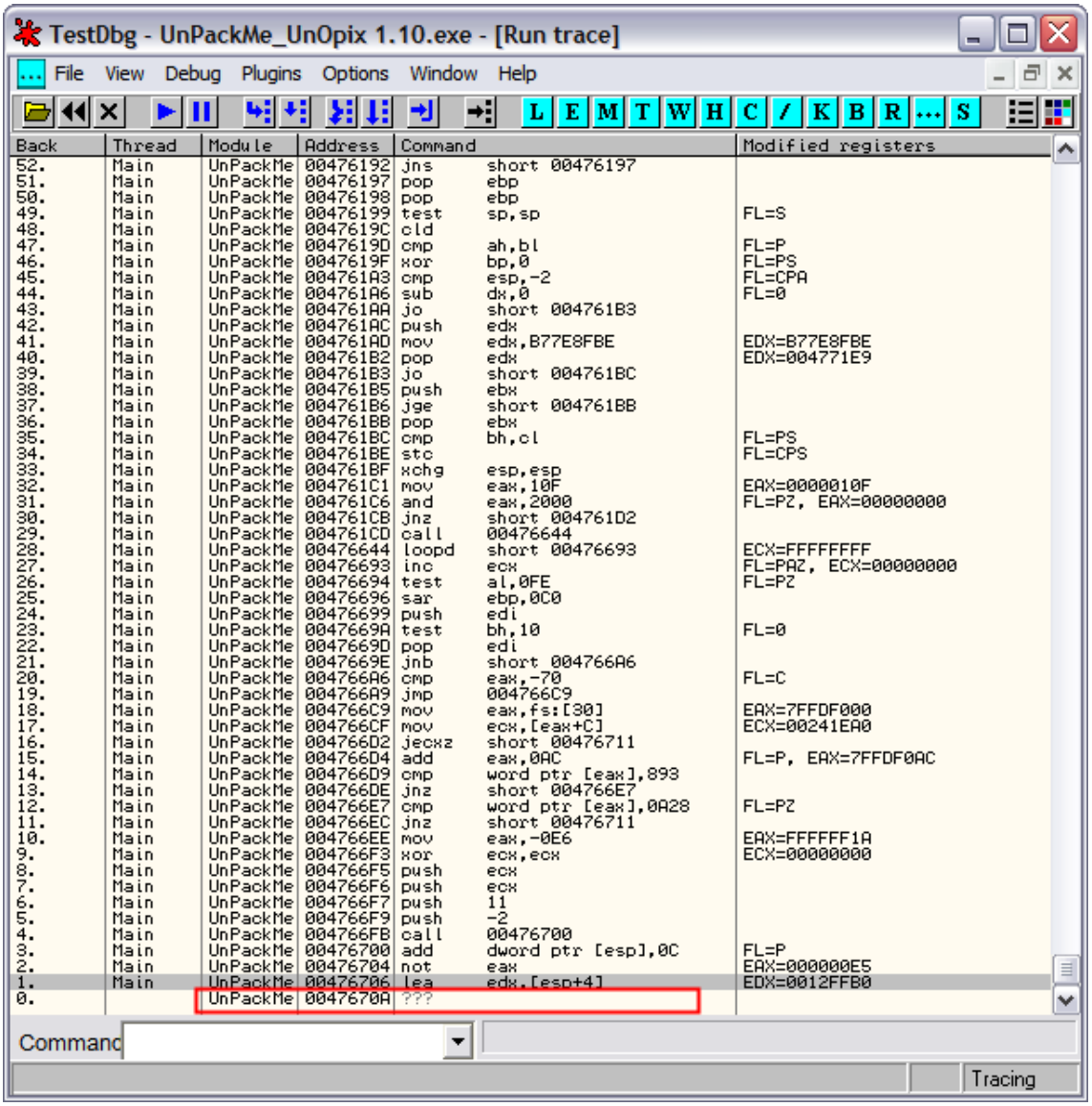
Cambiando de táctica..

Eso es lo que haremos, podríamos probar a cargar la VMWare y/o trabajar con WINDBG pero no quiero, lo voy a intentar igual con Olly, está claro que debe de ser algún truco antidebug que esconde entre el código del packer, lo siguiente que haré será tratar de hacer una traza y luego mirarla :D

Vamos a ello, cargamos el crackme en Olly y detenidos en el EP picamos el botoncito de los puntos suspensivos:

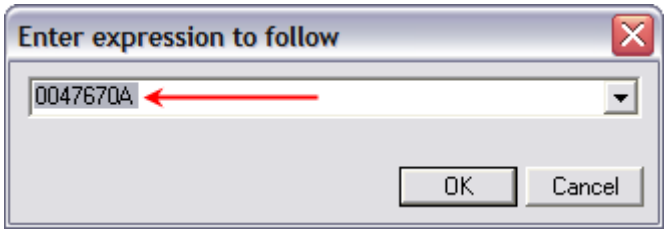
Se abre otra ventanita donde picamos con el botón derecho y elegimos LOG TO FILE, le damos un nombre como TRAZA.TXT y dejamos correr Olly con CTRL+F11 (Trace Into) para que entre en los CALL, Olly quedará así

Esto significa que está ejecutando código con F7 y escribiendo en el TXT cada línea que ejecuta (esto es el trazador que Olly incorpora). Veremos si en esta ocasión da resultado, cuando veamos que el código desaparece, así como la zona de datos (dump) y de pila (stack) es el momento de cerrar la traza. Pulsamos el mismo botón de antes de los puntos suspensivos y picamos con el botón derecho y CLOSE LOG FILE. Ya está, ya tenemos los datos de lo que ejecutó, el archivo resultante es bastante pesado 28,2 MB esto es porque ejecutó bastante código. El mismo Olly nos da la aclaración de lo que ejecutó:



Vemos que ejecutó código normalmente hasta llegar a :0047670A donde debe haber algo que le hace perder los papeles.

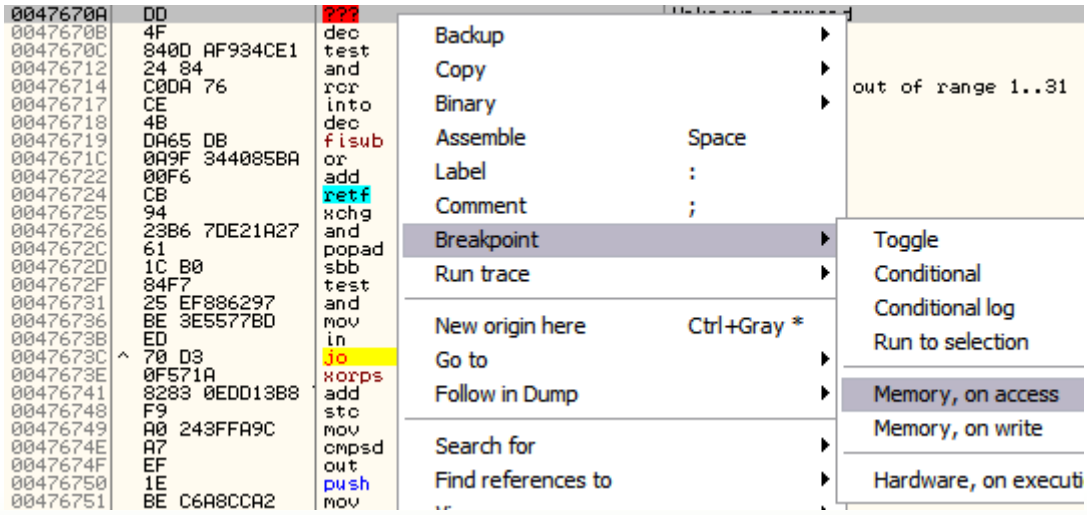
Cerremos Olly con ALT+X y aceptamos el cartel de error, lo volvemos a abrir y desde el EP hacemos un GOTO->EXPRESSION y ponemos esto:



Pulsamos OK y nos lleva a la zona, pero está toda encriptada como se puede ver en la siguiente imagen:

```
0047670A   DD                ???                                  Unknown command
0047670B   4F                dec     edi
0047670C   840D AF934CE1     test    [E14C93AF],cl
00476712   24 84             and     al,84
00476714   C0DA 76           rcr     dl,76                        Shift constant out of :
00476717   CE                into
00476718   4B                dec     ebx
00476719   DA65 DB           fisub   dword ptr [ebp-25]
0047671C   0A9F 344085BA     or      bl,[edi+BA854034]
00476722   00F6              add     dh,dh
00476724   CB                retf                                 Far return
00476725   94                xchg    eax,esp
00476726   23B6 7DE21A27     and     esi,[esi+271AE27D]
0047672C   61                popad
0047672D   1C B0             sbb     al,0B0
0047672F   84F7              test    bh,dh
00476731   25 EF886297       and     eax,976288EF
00476736   BE 3E5577BD       mov     esi,BD77553E
0047673B   ED                in      eax,dx                       I/O command
0047673C ^ 70 D3             jo      short 00476711
0047673E   0F571A            xorps   xmm3,[edx]
00476741   8283 0EDD13B8     add     byte ptr [ebx+B813DD
00476748   F9                stc
00476749   A0 243FFA9C       mov     al,[9CFA3F24]
0047674E   A7                cmpsd
0047674F   EF                out     dx,eax                       I/O command
00476750   1E                push    ds
00476751   BE C6A8CCA2       mov     esi,A2CCA8C6
00476756   0E                push    cs
00476757   49                dec     ecx
00476758   35 AFF30CCD       xor     eax,CD0CF3AF
0047675D   0D 35FC63DA       or      eax,DA63FC35
00476762   2D CB5AB90E       sub     eax,0EB95ACB
00476767   A2 78AA8CC7       mov     [C78CAA78],al
0047676C   CB                retf                                 Far return
0047676D   6247 B2           bound   eax,[edi-4E]
```

La línea perseguida tiene los bytes 0xDDh en este momento, vamos a ponerle un Breakpoint, picamos sobre ella con el botón derecho y seleccionamos un BP on Memory Access:



Pulsamos ahí, damos RUN y Olly en seguida para aquí:

```
0047744B   8B50 04        mov    edx,[eax+4]      <-- Está leyendo..
0047744E   0FCA           bswap  edx
00477450   89D6           mov    esi,edx
00477452   BA 000077E3    mov    edx,E3770000
00477457   50             push   eax
00477458   52             push   edx
00477459   C1F8 E0        sar    eax,0E0          Shift constant out of range 1..31
0047745C   5A             pop    edx
0047745D   58             pop    eax
0047745E   C1FE 80        sar    esi,80           Shift constant out of range 1..31
00477461   C1EB 20        shr    ebx,20           Shift constant out of range 1..31
00477464   FC             cld
00477465   50             push   eax
00477466 v 7F 03          jg     short 0047746B
00477468   83C0 00        add    eax,0
0047746B   58             pop    eax
0047746C v 71 03          jno    short 00477471
0047746E   83EA 00        sub    edx,0
00477471 v 7E 07          jle    short 0047747A
00477473 v 73 05          jnb    short 0047747A
00477475   52             push   edx
00477476   C0C5 F8        rol    ch,0F8           Shift constant out of range 1..31
00477479   5A             pop    edx
```

La barra de estado nos confirma que está leyendo de allí:

```
Memory breakpoint when reading [0047670A]                    Paused
```

Como no está ejecutando seguimos.. apretamos F9 otra vez y vuelve a parar, esta vez en otra dirección:

```
0047754A   8950 04        mov    [eax+4],edx      <-- Va a escribir..
0047754D   5A             pop    edx
0047754E   59             pop    ecx
0047754F   52             push   edx
00477550   6A F6          push   -0A
00477552   5A             pop    edx
00477553   5A             pop    edx
00477554   57             push   edi
00477555   C1F5 00        sal    ebp,0            Shift constant out of range 1..31
00477558   5F             pop    edi
00477559   56             push   esi
0047755A v 70 06          jo     short 00477562
0047755C v 7E 04          jle    short 00477562
0047755E   66:83CF 00     or     di,0
```

La barra de estado nos lo vuelve a confirmar:

```
Memory breakpoint when writing to [0047670A]                 Paused
```

Pues nada, tampoco es lo que buscamos, apretamos F9 y vuelve a parar:

```
0047670A   CD 2E          int    2E               <-- hummmmmm...
0047670C   83C4 14        add    esp,14
0047670F v EB 00          jmp    short 00476711
00476711   83C4 04        add    esp,4
00476714   8B0424         mov    eax,[esp]
00476717   3D 00000070    cmp    eax,70000000
0047671C v 7C 41          jl     short 0047675F
0047671E   E8 68050000    call   00476C8B
00476723   3D 00000070    cmp    eax,70000000
00476728 v 7C 35          jl     short 0047675F
```

Ahora sí estamos donde queríamos, jeje.. ese INT2E vuelve loco al Olly, grrrrrrrr.. buscando por Google algo de info encontré esto:

Subsections

- C.1 Application- and I/O-Level Views of the Data Stream
- C.2 Internal Data Structures (all are node-private)
  - C.2.1 Cache
- C.3 Subprograms
  - C.3.1 `sread`, `swrite` (in util directory)
  - C.3.2 `int2e_file_open` (API)

- C.3.2 int2e_file_open (API)
- C.3.3 int2e_file_close (API)
- C.3.4 int2e_file_rewind (API)
- C.3.5 int2e_file_read (API)
- C.3.6 int2e_file_write (API)
- C.3.7 int2e_file_write_big (internal)
- C.3.8 int2e_buf_read, int2e_buf_write (mostly internal)
- C.3.9 int2e_buf_clear (internal)
- C.3.10 int2e_buf_cntr_pack, int2e_buf_cntr_unpack (internal)
- C.3.11 int2e_buf_pack, int2e_buf_unpack (internal)
- C.3.12 int2e_set_bf_range, int2e_get_bf_range (API)

C. Integral File I/O - INT2E

C.1 Application- and I/O-Level Views of the Data Stream

The data stream coming into the package from the application consists of (floating point) integral values and (integer) labels (four labels per value) interspersed with calls which specify the ranges of the four labels for subsequent integrals.

Y bueno una parrafada de la que no cojo ni papa ?? pero bueno al menos lo intenté, aunque eso si, de poco me sirvió ;(

Seguí la búsqueda y di con algo más entendible:

INT 2E - Execute Command Using Base Level Command Interpreter (Undocumented DOS 2.0)

DS:SI = pointer to command string to be executed

returns nothing ???

- causes the base level COMMAND.COM to execute a command as if it

were typed from the keyboard

- the transient portion of COMMAND.COM must be loaded if not resident

- first byte of the string contains the string length, inclusive

- command string must be terminated with a CR

- all registers including SS & SP are destroyed; SS & SP should be

preserved in your own code segment

- can be used to modify parents environment with SET command

- incompatible under some software multitasking systems

- before returning to the parent program this interrupt will

attempt to execute the "current" batch file (if the parent or

any ancestors were invoked from a batch file. This can cause

remaining lines in a batch file to execute before the parent

program finishes and fragment memory

En cualquier caso sigo sin comprender bien su utilización, u todavía menos en un crackme, jeje.. está claro que el autor del mismo lo hizo para confundir al debugger de alguna manera y de hecho lo consigue, pues si lo ejecutamos fuera de Olly si que funciona.

Otra posibilidad sería probar con WINDBG pero la fiaca no me da como para ponerme a buscarlo entre la jauría de carpetas que tengo y luego instalarlo, etc, etc..

Inclusive encontré otro enlace:

http://www.delorie.com/djgpp/doc/rbinter/id/65/42.html

donde no decía demasiado ☹

*******//
Category: **other operating systems**
Flags: Undocumented function, available only in protected mode
INT 2E - Windows NT - NATIVE API

```
        EAX = function number (see #02586)
        EDX = address of parameter block
Return: ???
```

Si pico en el enlace #02586 me lleva a una larga lista de parámetros:

```
)
Values for Windows NT NTOS function number:
 000h   AcceptConnectPort      (24 bytes of parameters)
 001h   AccessCheck            (32 bytes of parameters)
 002h   AccessCheckAndAuditAlarm       (44 bytes of parameters)
 003h   AddAtom                (8 bytes of parameters)
 004h   AdjustGroupsToken      (24 bytes of parameters)
 005h   AdjustPrivilegesToken  (24 bytes of parameters)
 006h   AlertResumeThread      (8 bytes of parameters)
 007h   AlertThread            (4 bytes of parameters)
 008h   AllocateLocallyUniqueId        (4 bytes of parameters)
 009h   AllocateUuids          (12 bytes of parameters)
 00Ah   AllocateVirtualMemory  (24 bytes of parameters)
 00Bh   CallbackReturn         (12 bytes of parameters)
 00Ch   CancelIoFile           (8 bytes of parameters)
 00Dh   CancelTimer            (8 bytes of parameters)
 00Eh   ClearEvent             (4 bytes of parameters)
 00Fh   Close                  (4 bytes of parameters)
 010h   CloseObjectAuditAlarm  (12 bytes of parameters)
 011h   CompleteConnectPort    (4 bytes of parameters)
```

Y así hasta 0xD3h, ughhhh que mala pinta tiene, tendría que estar un buen rato estudiando TODO esto para saber exactamente QUÉ hace esta interrupción, así que de momento pasaremos al atake por otro lado ☺

Buscando soluciones...

Pues eso es, dado que no soy programador y apenas entiendo inglés esa info no me ayuda demasiado que se diga, entonces me dejo de historias y paso a otros métodos, probaré formas de superar ese INT2E como sea y que Olly pueda continuar ejecutando código a partir de él inclusive probando con el método universal de "ensayo y error" si hiciera falta.

Lo que hice en primer lugar fue NOPearlo:

```
0047670A   90              nop
0047670B   90              nop          <-- INT2E NOPeado!
0047670C   83C4 14         add     esp,14
0047670F ~ EB 00           jmp     short 00476711
00476711   83C4 04         add     esp,4
00476714   8B0424          mov     eax,[esp]
00476717   3D 00000070     cmp     eax,70000000
0047671C ~ 7C 41           jl      short 0047675F
0047671E   E8 68050000     call    00476C8B
```

Y al dar RUN Olly paraba aquí:

```
0047688C   8990 B8000000   mov     [eax+B8],edx      UnPackMe.004768A6
00476892   31D2            xor     edx,edx
00476894   8950 04         mov     [eax+4],edx
00476897   8950 08         mov     [eax+8],edx
0047689A   8950 0C         mov     [eax+C],edx
0047689D   8950 10         mov     [eax+10],edx
004768A0   31C0            xor     eax,eax
004768A2   48              dec     eax
004768A3   C2 0400         ret     4
```

Si miro la barra de estado veo:

```
Access violation when writing to [000000B8] - use Shift+F7/F8/F9 to pass ex   Paused
```

Si la paso con SHIFT+F9 Olly muere ☹

Tras muchas pruebas NADA me daba resultado, estaba claro que ese INT2E hace que el debugger busque o espere algo que no encuentra, al final hablando con dapaf vía MSN al que le envío un gran abrazo (gracias amigo) me comentó que había una forma bien sencilla de correr el crackme bajo Olly, así que utilizaré ese método, pongo un HE en el INT2E para parar allí cada vez que reinicie Olly (por si acaso). Luego llegao hasta él y me detengo:

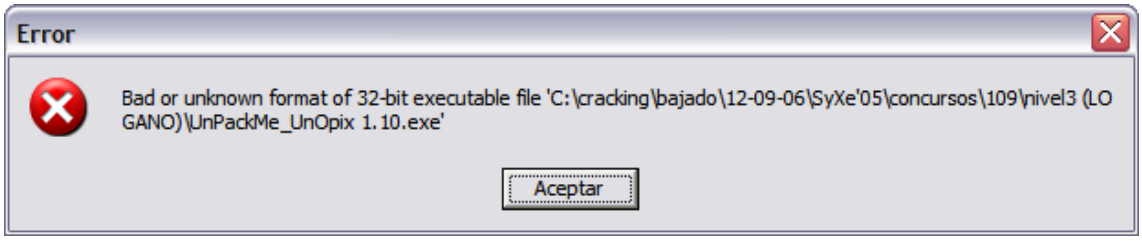
```
0047670A   CD 2E           int     2E           <-- hummmmmm...
0047670C   83C4 14         add     esp,14
0047670F ~ EB 00           jmp     short 00476711
00476711   83C4 04         add     esp,4
00476714   8B0424          mov     eax,[esp]
00476717   3D 00000070     cmp     eax,70000000
0047671C ~ 7C 41           jl      short 0047675F
0047671E   E8 68050000     call    00476C8B
00476723   3D 00000070     cmp     eax,70000000
00476728 ~ 7C 35           jl      short 0047675F
```

La cuestión es saltear ese INT2E como sea, por alguna razón si hago un NEW ORIGIN HERE en la siguiente línea 0x0047670C Olly muere, y dapaf me avisó que haciéndolo en el JMP sí que corría.

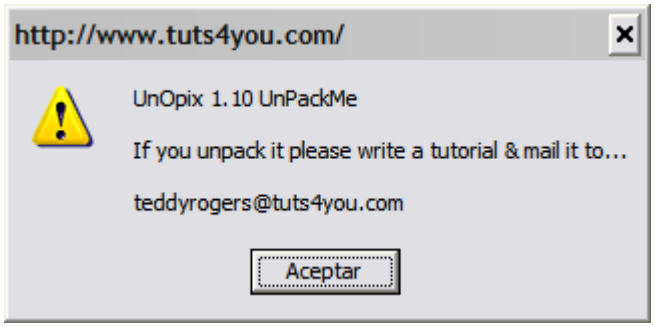
```
0047670A   CD 2E          int     2E
0047670C   83C4 14        add     esp,14
0047670F   EB 00          jmp     short 00476711
00476711   83C4 04
00476714   8B0424
00476717   3D 00000070
0047671C   7C 41
0047671E   E8 68050000
00476723   3D 00000070
00476728   7C 35
0047672A   89C3
0047672C   E8 1C000000
00476731   53
00476732   65:74 55
00476735   6E
00476736   68 616E646C
0047673B   65:
0047673C   64:45
0047673E   78 63
00476740   65:70 74
00476743   696F 6E 466960
0047674A   65:72 00
0047674D   58
0047674E   E8 5F050000
```

Backup
Copy
Binary
Assemble Space
Label :
Comment ;
Breakpoint
Run trace
Follow Enter
New origin here Ctrl+Gray *
Go to

Y prosigo la ejecución a partir de 0x0047670F y apreto F9, sale esto:



Lo acepto y aparece el crackme, jejejeje..



Buuuuuuuuuuuuuffff, menos mal, finalmente corre y como curiosidad os diré que aún NOPeando el INT2E más la siguiente línea del ADD ESP,14 no funciona, pero.. porqué? Pues porque aparentemente el autor ha agregado una protección del tipo CRC32 (o similar), esto se descubre NOPeando ambas líneas y colocando un BPM ON ACCESS en los 0x90, de esta manera se descubre el pastel:

```
00476EA5   C1EF 10        shr     edi,10
00476EA8   31D2           xor     edx,edx
00476EAA   E3 25          jecxz   short 00476ED1
00476EAC   8A10           mov     dl,[eax]          Zona CRC32
00476EAE   01D7           add     edi,edx
00476EB0   81FF F1FF0000  cmp     edi,0FFF1
00476EB6   7C 06          jl      short 00476EBE
00476EB8   81EF F1FF0000  sub     edi,0FFF1
00476EBE   01FE           add     esi,edi
00476EC0   81FE F1FF0000  cmp     esi,0FFF1
00476EC6   7C 06          jl      short 00476ECE
00476EC8   81EE F1FF0000  sub     esi,0FFF1
00476ECE   40             inc     eax
00476ECF   E2 DB          loopd   short 00476EAC
00476ED1   5A             pop     edx
00476ED2   C1E6 10        shl     esi,10
00476ED5   01F7           add     edi,esi
00476ED7   893A           mov     [edx],edi
00476ED9   5E             pop     esi
00476EDA   5F             pop     edi
00476EDB   C3             ret
```

Ahí esta leyendo dichos bytes, jeje.. y después salva el resultado:

```
00476EA5   C1EF 10          shr     edi,10
00476EA8   31D2             xor     edx,edx
00476EAA v E3 25            jecxz   short 00476ED1
00476EAC   8A10             mov     dl,[eax]                Zona CRC32
00476EAE   01D7             add     edi,edx
00476EB0   81FF F1FF0000    cmp     edi,0FFF1
00476EB6 v 7C 06            jl      short 00476EBE
00476EB8   81EF F1FF0000    sub     edi,0FFF1
00476EBE   01FE             add     esi,edi
00476EC0   81FE F1FF0000    cmp     esi,0FFF1
00476EC6 v 7C 06            jl      short 00476ECE
00476EC8   81EE F1FF0000    sub     esi,0FFF1
00476ECE   40               inc     eax
00476ECF ^ E2 DB            loopd   short 00476EAC
00476ED1   5A               pop     edx
00476ED2   C1E6 10          shl     esi,10
00476ED5   01F7             add     edi,esi
00476ED7   893A             mov     [edx],edi               salva resultado!
00476ED9   5E               pop     esi
00476EDA   5F               pop     edi
00476EDB   C3               ret
```

Ahí está, lo podemos ver en el dump:

```
Address   Hex dump                                          ASCII
004771D0  0F 72 94 B2 60 22 BB 2F E5 AF 69 87 7E 07 F3 D2
004771E0  00 00 00 00 90 90 90 90 90 6C 17 0B BC C8 50 E7
004771F0  A9 21 A9 B3 87 AA D5 C7 A2 A8 68 F9 0B E4 81 FE
00477200  BA 97 44 91 7B 04 66 83 F8 EE 22 D2 76 07 56 A9
00477210  62 01 CF 00 Resultado CRC32 47 5F 5B 87 D2 7F
00477220  03 57 47 5F 73 06 55 7A 04 66 83 F4 00 5D 53 50
00477230  76 03 C0 CB F8 58 5B C0 C5 80 51 66 23 C9 59 78
00477240  05 73 03 C1 C3 E0 71 05 6A 89 83 C4 04 F5 FC 53
00477250  81 FA 5E 41 4F 7F 5B C1 EE 60 F8 56 F7 C2 99 93
00477260  1E 9E 5E 55 83 F9 65 5D FC 57 75 06 57 66 C1 C0
```

Ahí está, quedó 0xB294720F (creo que esta captura esta tomada con la zona modificada ahora no estoy seguro) pero no importa pues el método es este, después retorna al CALL padre desde donde hizo la llamada y le hace un XOR al resultado del CRC, siguiendo este hilo se puede localizar la comparación y saltearla, o quizá desencripte código con él pero como no indagué más no llegué a verlo puesto que el crackme ya corre en Olly y con esto es más que suficiente ;)

Rematando la faena..

Así es señores, como ya sabemos hacer correr el crackme en Olly es cuestión de plantarnos en el OEP y dumpear, nos quedamos cuando hicimos el HEW ORIGIN HERE hacia 0x0047670F, nos vamos al mapa de memoria con ALT+M y le ponemos un BPM ON MEMORY ACCESS a la sección de código (la siguiente al PE header) y damos RUN, Olly para aquí:

```
0046DE9B   8807             mov     [edi],al
0046DE9D   47               inc     edi
0046DE9E   01DB             add     ebx,ebx
0046DEA0 v 75 07            jnz     short 0046DEA9
0046DEA2   8B1E             mov     ebx,[esi]
0046DEA4   83EE FC          sub     esi,-4
0046DEA7   11DB             adc     ebx,ebx
0046DEA9 ^ 72 ED            jb      short 0046DE98
0046DEAB   B8 01000000      mov     eax,1
0046DEB0   01DB             add     ebx,ebx
0046DEB2 v 75 07            jnz     short 0046DEBB
0046DEB4   8B1E             mov     ebx,[esi]
0046DEB6   83EE FC          sub     esi,-4
0046DEB9   11DB             adc     ebx,ebx
0046DEBB   11C0             adc     eax,eax
0046DEBD   01DB             add     ebx,ebx
0046DEBF v 73 0B            jnb     short 0046DECC
0046DEC1 v 75 19            jnz     short 0046DEDC
0046DEC3   8B1E             mov     ebx,[esi]
0046DEC5   83EE FC          sub     esi,-4
0046DEC8   11DB             adc     ebx,ebx
0046DECA v 72 10            jb      short 0046DEDC
0046DECC   48               dec     eax
0046DECD   01DB             add     ebx,ebx
0046DECF v 75 07            jnz     short 0046DED8
0046DED1   8B1E             mov     ebx,[esi]
```

Parece el típico loop desencriptador, parece que vamos bien, paró por ESCRITURA (on writting), en este momento EDI==401000 y si vamos dando

RUN irá aumentando de a 1 y parando en esta zona, entonces quitamos provisionalmente el BPM y encontramos un POPAD, le colocamos un BP:

```
0046DFCD   47              inc     edi
0046DFCE   B9 5748F2AE     mov     ecx,AEF24857
0046DFD3   55              push    ebp
0046DFD4   FF96 18490700   call    [esi+74918]
0046DFDA   09C0            or      eax,eax
0046DFDC v 74 07           je      short 0046DFE5
0046DFDE   8903            mov     [ebx],eax
0046DFE0   83C3 04         add     ebx,4
0046DFE3 ^ EB D8           jmp     short 0046DFBD
0046DFE5   FF96 1C490700   call    [esi+7491C]
0046DFEB   61              popad   <---- jeje un POPAD ;)
0046DFEC v E9 668D0000     jmp     00476D57
0046DFF1   0000            add     [eax],al
0046DFF3   0000            add     [eax],al
0046DFF5   0000            add     [eax],al
```

Damos RUN y Olly se detiene por el BP que pusimos:

```
0046DFDA   09C0            or      eax,eax
0046DFDC v 74 07           je      short 0046DFE5
0046DFDE   8903            mov     [ebx],eax
0046DFE0   83C3 04         add     ebx,4
0046DFE3 ^ EB D8           jmp     short 0046DFBD
0046DFE5   FF96 1C490700   call    [esi+7491C]
0046DFEB   61              popad
0046DFEC v E9 668D0000     jmp     00476D57
0046DFF1   0000            add     [eax],al
0046DFF3   0000            add     [eax],al
```

Quitamos el BP por si acaso y pasamos el JMP que nos lleva a otra rutina, la vamos traceando y vemos que revisa que no haya ningún BP en el OEP en esta línea:

```
00476E19   F5              cmc
00476E1A   50              push    eax
00476E1B   66:B8 12E5      mov     ax,0E512
00476E1F   58              pop     eax
00476E20   84ED            test    ch,ch
00476E22   8038 CC         cmp     byte ptr [eax],0CC
00476E25 ^ 0F84 0BF8FFFF   je      00476636
00476E2B   F6D0            not     al
00476E2D   F6D0            not     al
00476E2F v 77 08           ja      short 00476E39
```

Si miramos los registros en este momento vemos:

```
Registers (FPU)
EAX 004271B0 <----
ECX 77F44DB4 ntdll.77F44DB4
EDX 77E614C7 kernel32.77E614C7
EBX 004761D2
ESP 0012FFD0
EBP 00000000
ESI 00000011
EDI FFFFFFFE
```

Sabemos que el UnPackMe de Teddy Rogers tiene este OEP==4271B0, como no hay BP continúa sin tomar el JE, esto nos va bien porque no mucho después nos encontramos con el salto al OEP ;)

```
00476E6A   58              pop     eax
00476E6B v 76 02           jbe     short 00476E6F
00476E6D   85C0            test    eax,eax
00476E6F   40              inc     eax
00476E70   48              dec     eax
00476E71   53              push    ebx
00476E72   83C4 04         add     esp,4
00476E75 ^ FFE0            jmp     eax                 EAX==4271B0
00476E77 v 7F 05           jg      short 00476E7E
00476E79   05 00000000     add     eax,0
```

Lo seguimos y..

```
004271B0  90              nop                                  y el PUSH EBP ??
004271B1  8BEC            mov     ebp,esp
004271B3  6A FF           push    -1
004271B5  68 600E4500     push    450E60
004271BA  68 C8924200     push    4292C8
004271BF  64:A1 00000000  mov     eax,fs:[0]
004271C5  50              push    eax
004271C6  64:8925 000000  mov     fs:[0],esp
004271CD  83C4 A8         add     esp,-58
004271D0  53              push    ebx
004271D1  56              push    esi
004271D2  57              push    edi
004271D3  8965 E8         mov     [ebp-18],esp
004271D6  FF15 DC0A4600   call    [460ADC]                     kernel32.GetVersion
004271DC  33D2            xor     edx,edx
```

Ya estamos con cosas raras, ahí falta un PUSH EBP, si miramos el stack vemos que no empujó nada:

```
0012FFD0  7FFDF000  UnPack_1.7FFDF000
0012FFD4  B179ACF0
0012FFD8  0012FFC8
0012FFDC  8053C88F
0012FFE0  FFFFFFFF  End of SEH chain
0012FFE4  77E74809  SE handler                    Stack
0012FFE8  77E71210  kernel32.77E71210             equilibrado!
0012FFEC  00000000
0012FFF0  00000000
0012FFF4  00000000
0012FFF8  0040009C
0012FFFC  00000000
```

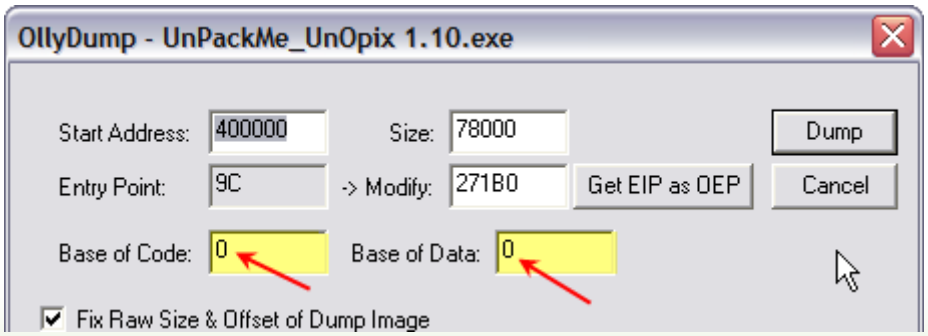
Para alucinar!!!! Pero bien será otro truco sucio ?? como ya no me fio ni un pelo le pongo yo mismo el PUSH EBP y listos:

```
004271B0  55              push    ebp
004271B1  8BEC            mov     ebp,esp
004271B3  6A FF           push    -1
004271B5  68 600E4500     push    450E60
004271BA  68 C8924200     push    4292C8
004271BF  64:A1 00000000  mov     eax,fs:[0]
004271C5  50              push    eax
004271C6  64:8925 000000  mov     fs:[0],esp
004271CD  83C4 A8         add     esp,-58
004271D0  53              push    ebx
004271D1  56              push    esi
004271D2  57              push    edi
004271D3  8965 E8         mov     [ebp-18],esp
004271D6  FF15 DC0A4600   call    [460ADC]                     kernel32.GetVersion
004271DC  33D2            xor     edx,edx
```

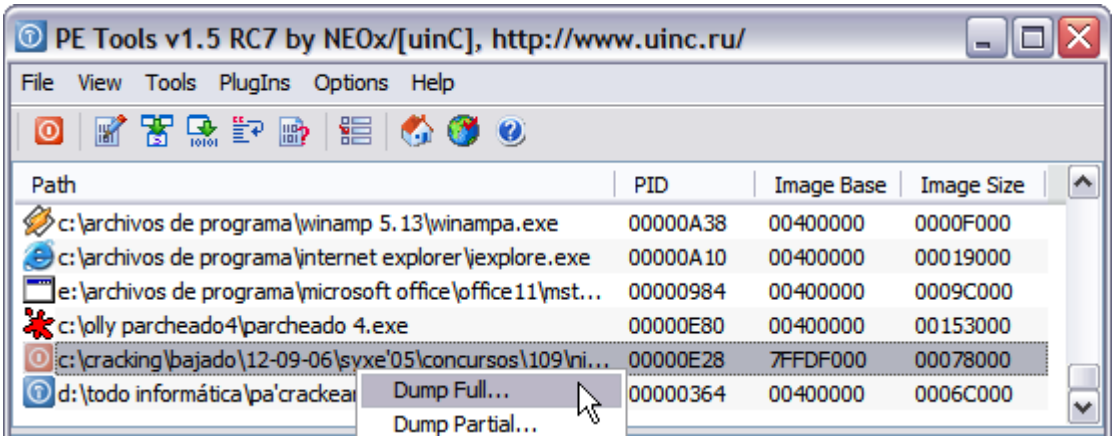
Ahora trataré de dumpear a ver si se puede porque ya antes vimos que se automodificó y destrozó la estructura de la cabecera, solo hay que mirar en el mapa de memoria y allí veremos que aparece todo el crackme en una única sección:

Address	Size	Owner	Section	Contains	Type	Access	Initial	Mappec
003A0000	00003000				Map	R	R	\Devic
003B0000	00002000				Map	R	R	
003C0000	00003000				Priv	RW	RW	
003D0000	00002000				Map	R	R	
00400000	00078000				Imag	R	RWE	
00480000	00001000	OLE32		PE header	Imag	R	RWE	
00481000	00104000	OLE32	.text	code,import	Imag	R	RWE	
00585000	00006000	OLE32	.orpc	code	Imag	R	RWE	
0058B000	00007000	OLE32	.data	data	Imag	R	RWE	
00592000	00002000	OLE32	.rsrc	resources	Imag	R	RWE	
00594000	0000D000	OLE32	.reloc	relocations	Imag	R	RWE	

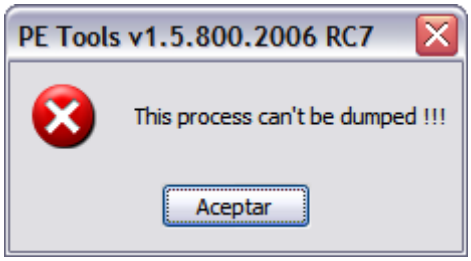
Si miramos de dumpear con OllyDump vemos que los campos BASE OF CODE y BASE OF DATA están ambos a cero:



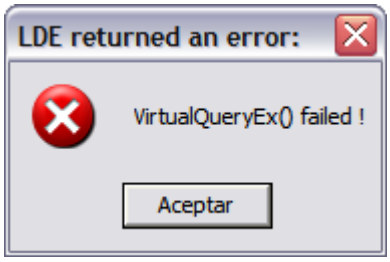
Se pueden editar a mano y no habría problema pero en estos casos es preferible dumpear con una herramienta externa tipo LordPE, PE Tools, etc.. aquí utilizaré PE Tools RC7:

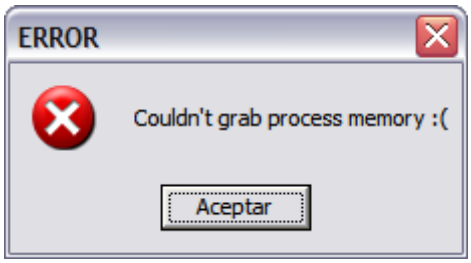


Pero me quedo con dos palmos de narices¡!!!!!!!!



Si lo intento con LordPE me pasa lo mismo:





Y bueno, pupe me deja un dumpeado de 0 KB y sin código (ni si quiera tiene icono), grrrrrrrrrrr… la cosa se está poniendo fea!!!

Hagamos una cosa, podríamos abrir el cracke con la cabecera aún sin modificar en otro Olly y hacer un binary copy/binary paste para copiar la cabecera correcta y tratar de dumpear, o bien modificar los valores BASE OF CODE y BASE OF DATA a mano en Olly y tratar de dumpear desde aquí mismo, probaré esto segundo que es más instructivo.

Dumpeando lo indumpeable ;)

Sip, eso haremos esto lo vamos a dumpear si o si, jeje.. abramos otro Olly y carguemos el UnPackMe en él, vamos al mapa de memoria y vemos los valores que necesitamos:

Address	Size	Owner	Section	Contains	Type	Access	Initial	Mapped
003A0000	00003000				Map	R	R	\Devic
003B0000	00002000				Map	R	R	
003C0000	00003000				Priv	RW	RW	
003D0000	00002000				Map	R	R	
00400000	00001000	UnPackMe		PE header	Imag	R	RWE	
00401000	00044000	UnPackMe	.data	code	Imag	R	RWE	
00445000	00029000	UnPackMe	.text	code	Imag	R	RWE	
0046E000	0000A000	UnPackMe	.rsrc	code,data,i	Imag	R	RWE	
00480000	00001000	OLE32		PE header	Imag	R	RWE	
00481000	00104000	OLE32	.text	code,import	Imag	R	RWE	
00585000	00006000	OLE32	.orpc	code	Imag	R	RWE	
0058B000	00007000	OLE32	.data	data	Imag	R	RWE	
00592000	00002000	OLE32	.rsrc	resources	Imag	R	RWE	
00594000	0000D000	OLE32	.reloc	relocations	Imag	R	RWE	
005B0000	00008000				Map	R E	R E	

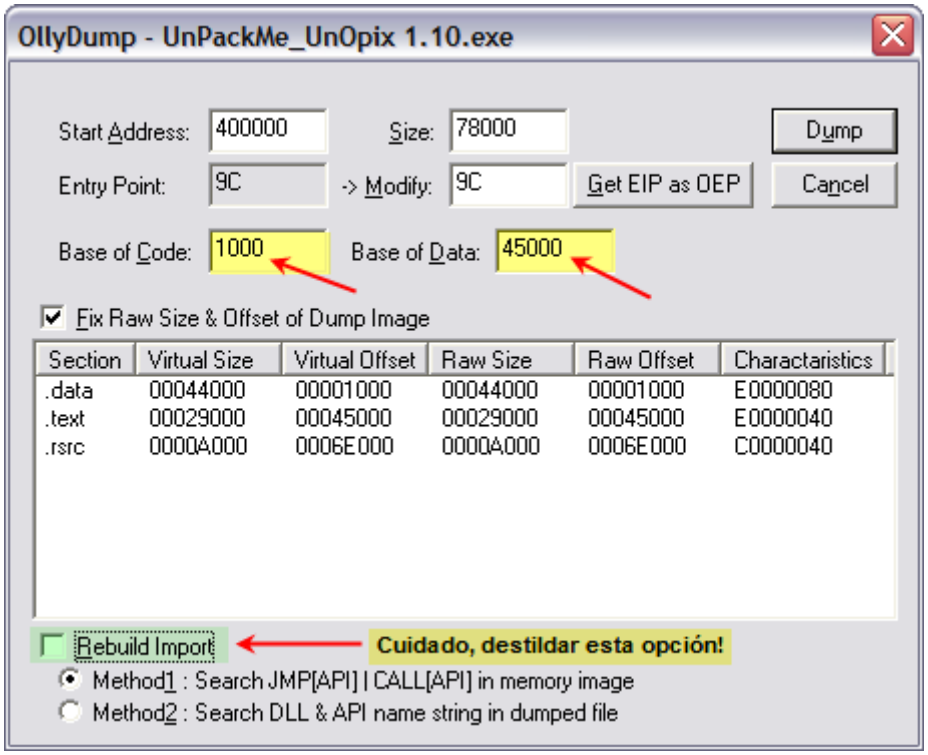
Vemos que el autor del crackme trata de despistar colocando la sección .text (la de código usualmente) en segundo lugar, además de nombrar ambas como CODE como se puede ver, pero la sección de código es la siguiente al PE header diga ahí lo que diga sip?

Tenemos pues:

Sección de código (CODE)
RVA: 401000 ->> offset: 1000

Sección de datos (DATA)
RVA: 445000 ->> offset: 45000

Estos dos valores son BASE OF CODE Y DATA respectivamente, los ponemos nosotros a mano en el primer Olly que tenemos abierto, este segundo ya lo podemos cerrar, ponemos los valores ya:

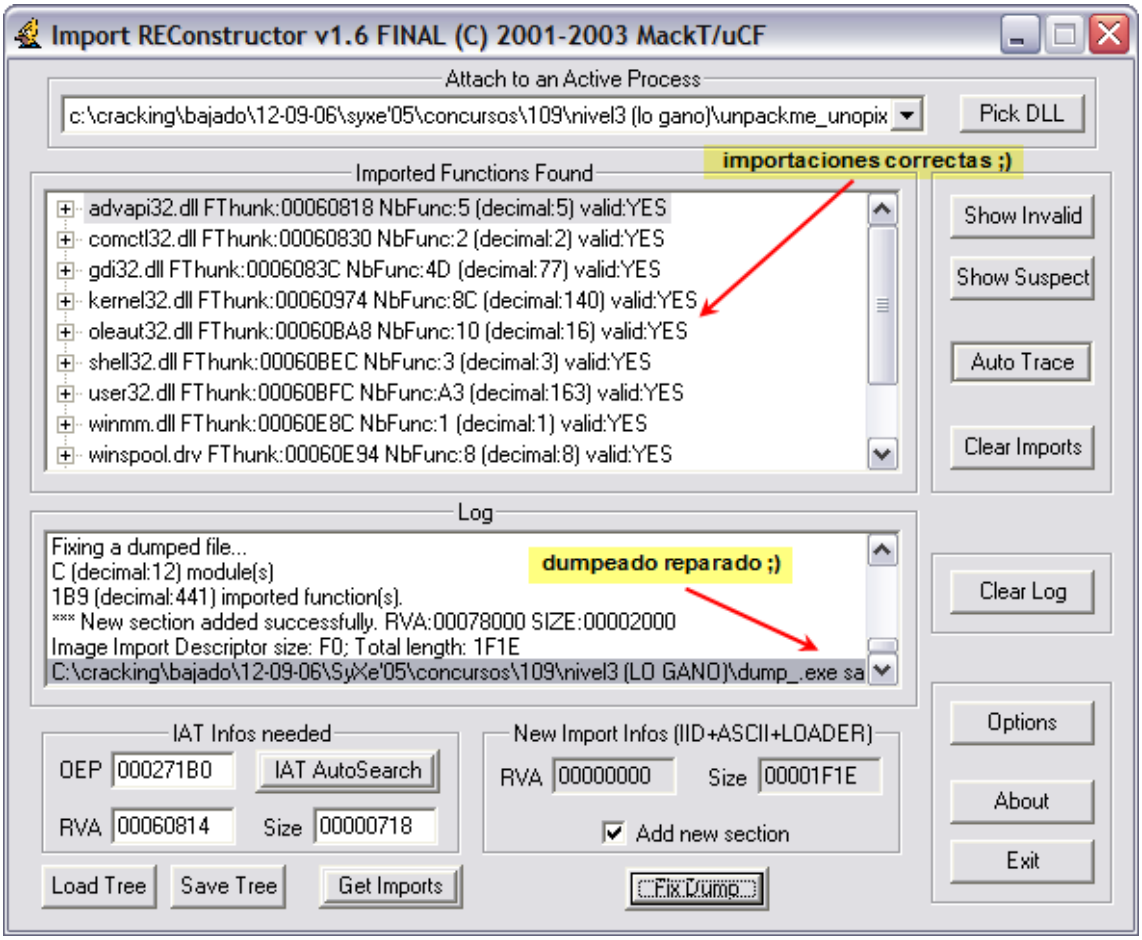


Cuidado de desmarcar la opción REBUILD IMPORT ya que luego lo arreglaremos con el ImpREC, picamos el GET EIP AS OEP y luego DUMP. Ya tenemos un dumpeado como Dios manda con icono y todo.

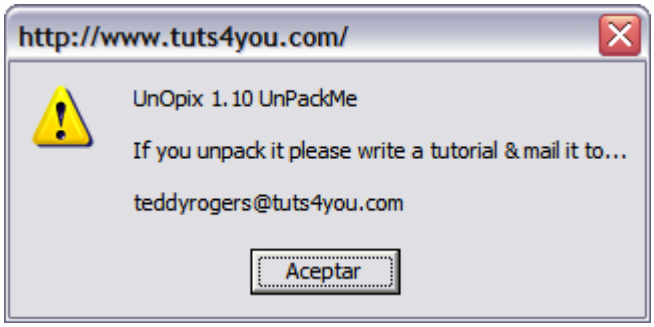
Abrimos el ImpREC 1.6f y seleccionamos el proceso de la lista donde están los procesos que están corriendo, le ponemos el offset del OEP:

OEP==4271B0 - 400000 (ImageBase) == 271B0

Y picamos en IAT AUTOSEARCH + GET IMPORTS y nos saca todas las APIs correctamente, reparemos el Dumpeado:



Hacemos doble clic sobre dump_.exe y...



Bueno prueba superada, jeje..

-** Saludos y/o Agradecimientos **-

Sólo me queda saludar a TODOS los componentes de CracksLatinoS SIN EXCEPCIONES, y en especial a mis grandes AMIGOS Ricardo, dapaf, Morales, solid, Mintaka, RedH@wk, Absolom, suei, emadicius, Juan Jose, k0rt, Carlos, txino, DeAth, GUAN, marciano y como no a Ulaterk, MairJr, c40s, Yllera, +NCR y a todo el grupo en GENERAL ;)

También por su puesto a todos aquellos de los que me olvido momentáneamente pero que aquí tienen su espacio XDD

Y sin más me despido hasta el próximo tute, un abrazo..

-** Contacto **-

Pueden contactarme rápidamente en la siguiente dirección de email:

syxe05@gmail.com

Un abrazo
SyXe'05